

Privacy Attacks on Stable Marriage

Stephan A. Fahrenkrog-Petersen

University of Liechtenstein

Aleksander Figiel
Darya Melnyk
Tijana Milentijević
Stefan Schmid

TU Berlin

Matching Problems Are Everywhere!

- assigning students to projects
- assigning residencies
- college admissions
- job markets
- donor-patient pairing
- ...

Matching Problems Are Everywhere!

- assigning students to projects
- assigning residencies
- college admissions
- job markets
- donor-patient pairing
- ...

Often: **privacy-sensitive** domains!

Matching is Sensitive: An Example (1)

- Example: US National [Resident Matching Program](#) (NRMP), aka “The Match”
- Matches [medical graduates](#) to [hospital residency](#) programs
- Matching solved [centrally](#): both applicants and clinics submit their preferences to the system

Matching is Sensitive: An Example (1)

- Example: US National **Resident Matching Program** (NRMP), aka “The Match”
- Matches **medical graduates** to **hospital residency** programs
- Matching solved **centrally**: both applicants and clinics submit their preferences to the system

What could go **wrong**?

Matching is Sensitive: An Example (2)

- **Hospitals** might **prefer** candidates who rank them highly, fearing that applicants who ranked them **lower would leave** after training
- Therefore, residency programs may have an incentive to learn applicants' **preferences** and exploit this information to their **advantage**.
- **Applicants** (graduates) are the more **vulnerable** side

Matching is Sensitive: Illustration

graduate A



$1 > 2 > 3$

graduate B



$2 > 1 > 3$

graduate C



$1 > 3 > 2$

Which graduate
really wants us
more?



clinic 1



clinic 2



clinic 3

The Threat and Need for Privacy

- Agents (e.g., hospitals) may aim to **infer** the users' **preference lists**
- Ensuring privacy hence critical to **prevent strategic manipulation**, discourage **misreporting**, and comply with data protection **regulations**

Our Paper

- Are matching algorithms **vulnerable** to privacy attacks?
- Can dishonest agents **lie about their preferences** to infer preferences of other (honest) agents?

Our Model (1)

- **Honest** and **dishonest** agents
- Preferences: given as **lists**
- **Bipartite** (e.g., graduates and hospitals)
- Dishonest agents **coordinate** and share information among each other
- Dishonest agents may **change their preferences** over time (and may not sure true values with central server)
- Focus on **stable matching** (marriage): no “couple” would prefer each other compared to their assigned partner

Our Model (2)

- $2n$ participants:

$H = \{h_1, h_2, \dots, h_n\}$ honest agents

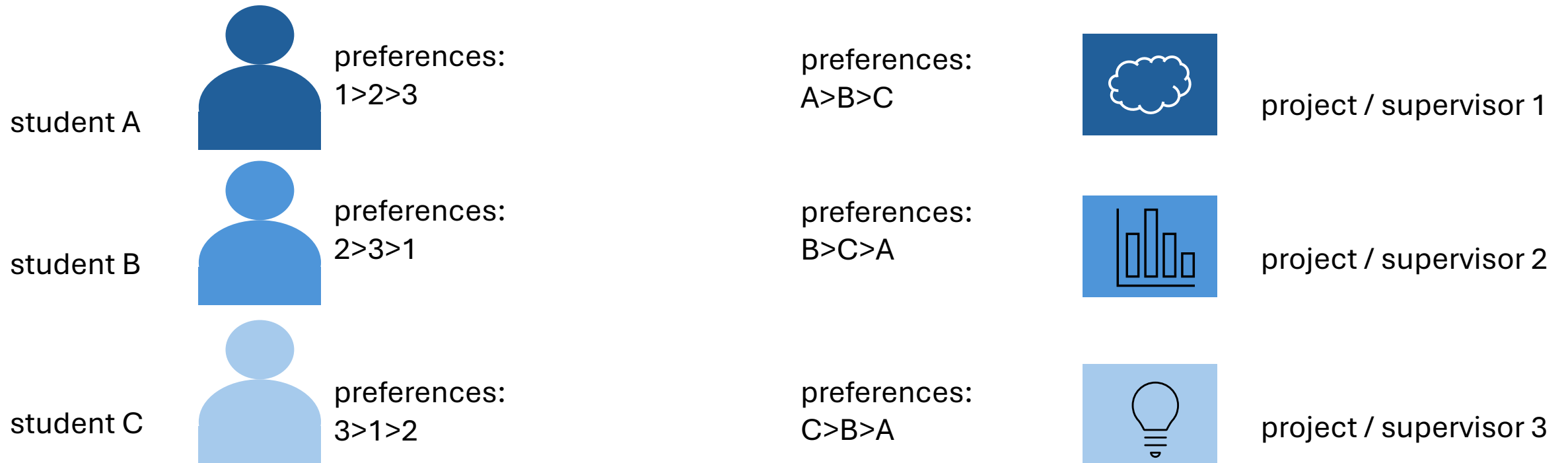
$D = \{d_1, d_2, \dots, d_n\}$ dishonest agents

- **Strict** preferences:

$b \succ_a c$ for agent $a \in H$ and $b, c \in D$

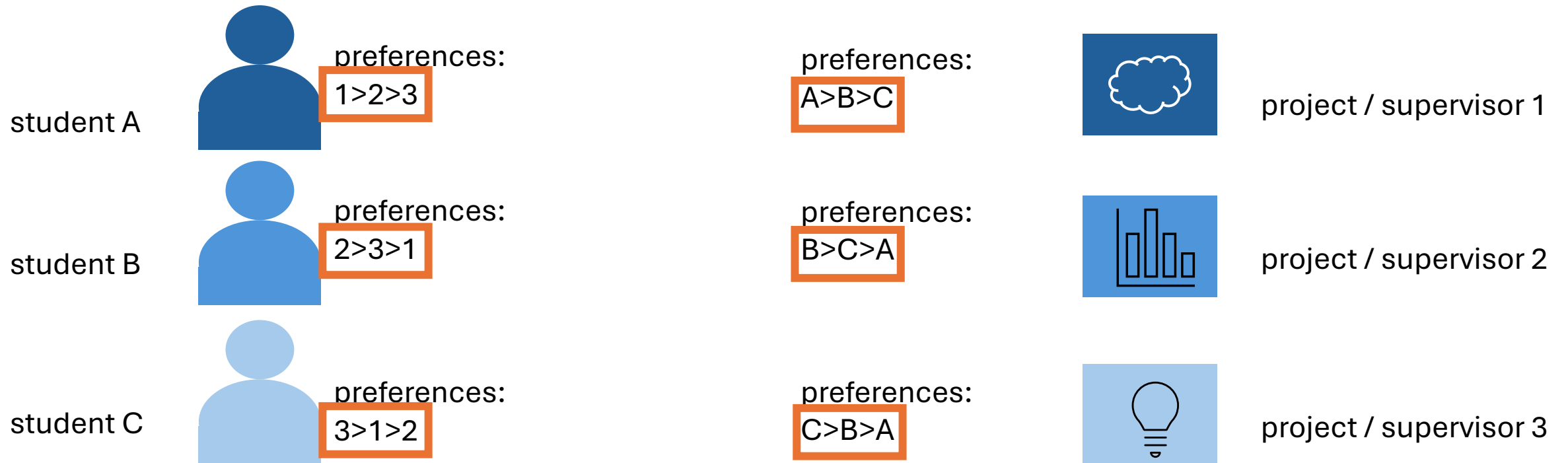
- Goal: find a **stable matching**

Illustration: Stable Matching



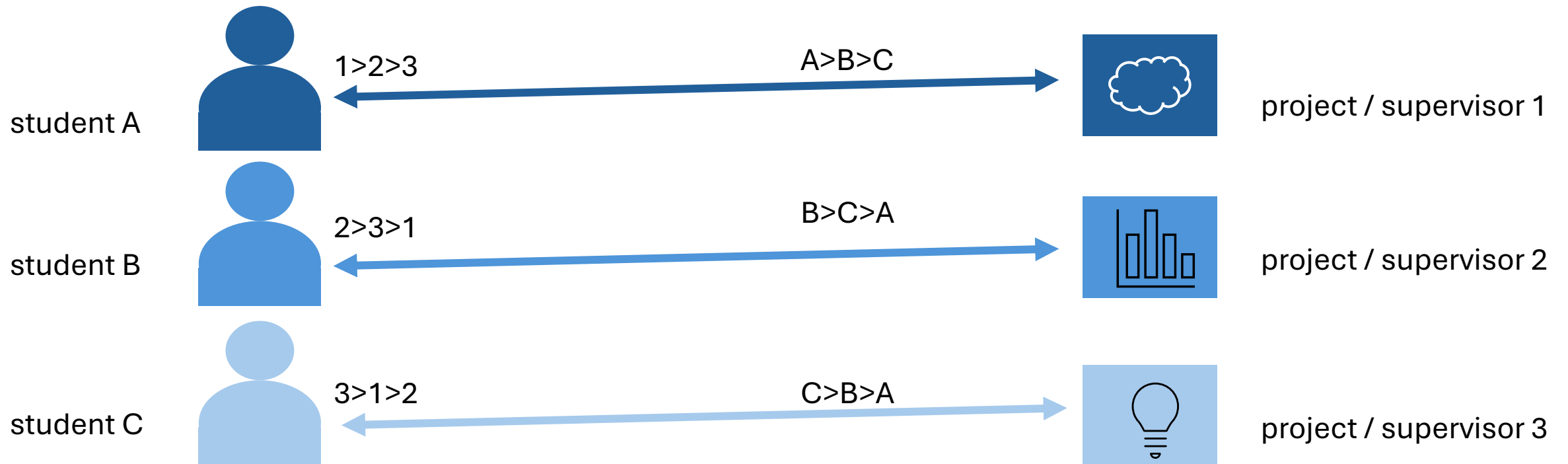
Example where agents have
different preference lists!

Illustration: Stable Matching



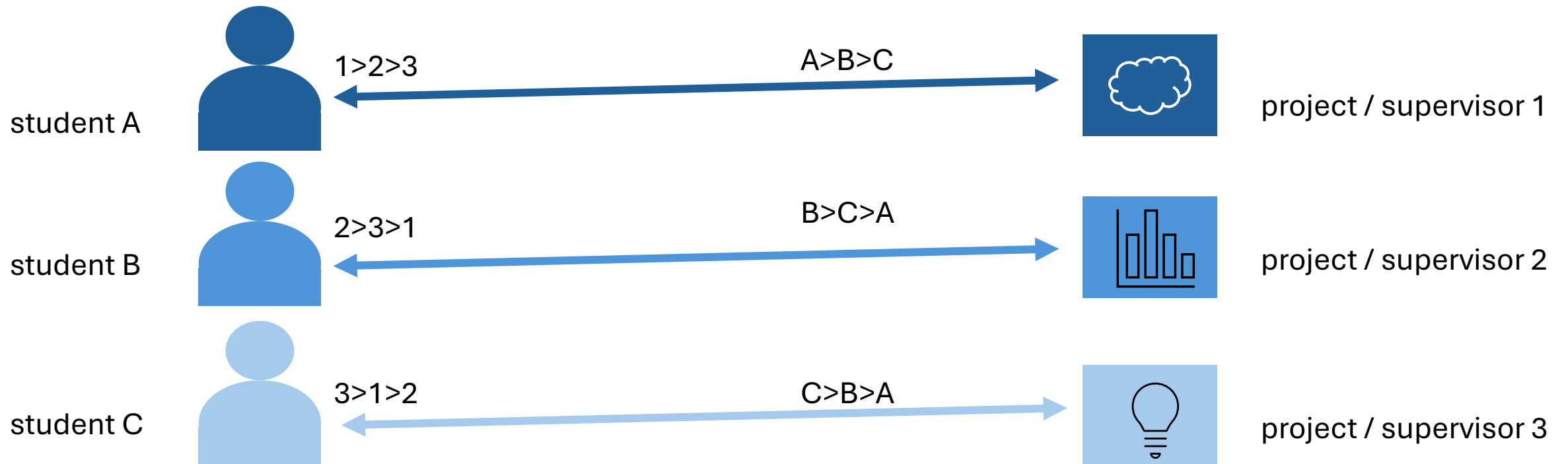
Example where agents have different preference lists!

Illustration: Stable Matching



This matching is **stable!** No student-supervisor pair would both prefer each other over their assigned partner.

Illustration: Stable Matching



This matching is **stable!** No student-supervisor pair would both prefer each other over their assigned partner.

How are stable matchings computed?

- For example the using the well-known **Gale-Shapley** algorithm
- Lets participants on **one side propose** to their top-ranked preference
- If rejected, they iteratively propose to the **next best** alternative in their ranking

Attack Example

Preferences of a,b:

a:  1 $\succ_a$ 2

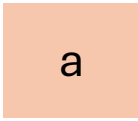
b:  2 $\succ_b$ 1

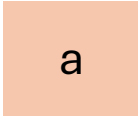
Attack Example

Preferences of a,b:

a:  $\succ_a 2$

b:  $\succ_b 1$

1:  $\succ_1 b$

2:  $\succ_2 b$

Scenario 1: claimed (lied)
preferences of 1,2

Dishonest agents 1,2 may first claim they both prefer a.

Attack Example

Preferences of a,b:

a: 1 $\succ_a$ 2

b: 2 $\succ_b$ 1

1: a $\succ_1$ b

2: a $\succ_2$ b

Scenario 1: claimed (lied) preferences of 1,2

Dishonest agents 1,2 may first claim they both prefer a.
So in any stable matching, a must be matched to its *first preference*.

Attack Example

Preferences of a,b:

a: 1 $\succ_a$ 2

b: 2 $\succ_b$ 1

1: a $\succ_1$ b

2: a $\succ_2$ b

Scenario 1: claimed (lied) preferences of 1,2

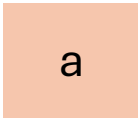
Dishonest agents 1,2 may first claim they both prefer a.
So in any stable matching, a must be matched to its *first preference*.
Since a was matched: 1 must be *first preference of a*!

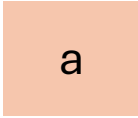
Attack Example

Preferences of a,b:

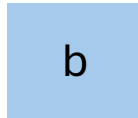
a:  $\succ_a 2$

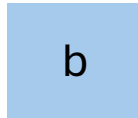
b:  $\succ_b 1$

1:  $\succ_1 b$

2:  $\succ_2 b$

Scenario 1: claimed (lied) preferences of 1,2

1:  $\succ_1 a$

2:  $\succ_2 a$

Scenario 2: claimed (lied) preferences of 1,2

Dishonest agents 1,2 then claim they both *prefer b*.

Attack Example

Preferences of a,b:

a: 1 $\succ_a 2$

b: 2 $\succ_b 1$

1: a $\succ_1 b$

2: a $\succ_2 b$

Scenario 1: claimed (lied) preferences of 1,2

1: b $\succ_1 a$

2: b $\succ_2 a$

Scenario 2: claimed (lied) preferences of 1,2

Dishonest agents 1,2 then claim they both prefer b.
So in any stable matching, **b** must be matched to its first preference.

Attack Example

Preferences of a,b:

a: 1 $\succ_a$ 2

b: 2 $\succ_b$ 1

1: a $\succ_1$ b

2: a $\succ_2$ b

Scenario 1: claimed (lied) preferences of 1,2

1: b $\succ_1$ a

2: b $\succ_2$ a

Scenario 2: claimed (lied) preferences of 1,2

Dishonest agents 1,2 then claim they both prefer b.
So in any stable matching, b must be matched to its first preference.
Since a was matched: 2 must be *first preference of b*!

Result 1: Can fine top prefs

- *Independent of the stable matching algorithm, it is **always possible** for the malicious side to learn **the top preference** of each agent in H within n iterations.*

a: 1 $\succ_a$ 2

b: 2 $\succ_b$ 1

round 1:

1: a $\succ_1$ b

2: a $\succ_2$ b

round 2:

1: b $\succ_1$ a

2: b $\succ_2$ a

- *Essentially a generalization of our example.*

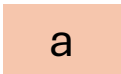
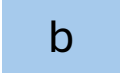
Result 2: Can find all prefs

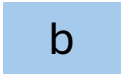
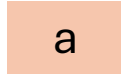
- If all good agents have **identical preferences**, then no stable matching algorithm can hide good agents' preferences*

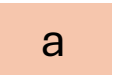
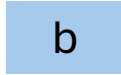
a:  $\succ_a$ 2 $\succ_a$ 3

b:  $\succ_b$ 2 $\succ_b$ 3

c:  $\succ_c$ 2 $\succ_c$ 3

round 1:  $\succ_{d_i}$  $\succ_{d_i}$  pairs: (a,1),(b,2),(c,3)

round 2:  $\succ_{d_i}$  $\succ_{d_i}$  (b,1),(c,2),(a,3)

round 3:  $\succ_{d_i}$  $\succ_{d_i}$  (c,1),(a,2),(b,3)

for all $d_i \in D$

Round Robin
Attack

- Position elimination strategy: in each iteration, every honest agent will be paired with a different malicious agent*

Result 3: Propose&Reject

- Using a P&R algorithm where dishonest agents propose, everything can be learned in n^2 rounds.

Result 3: Propose&Reject

- Using a P&R algorithm where **dishonest agents propose**, everything can be learned in n^2 rounds.
1. We can learn the first preference always! (Result 1)
 2. How to learn other preferences?

Result 3: Propose&Reject

- Using a P&R algorithm where **dishonest agents propose**, everything can be learned in n^2 rounds.

- We can learn the first preference always! (Result 1)
- How to learn other preferences?

a: $1 \succ_a ? \succ_a$

b: $2 \succ_b \succ_b$

c: $1 \succ_c \succ_c$

1: $b \succ_1 c \succ_1 a$

2: $a \succ_2 c \succ_2 b$

3: $a \succ_3 c \succ_3 b$

Result 3: Propose&Reject

- Using a P&R algorithm where **dishonest agents propose**, everything can be learned in n^2 rounds.

- We can learn the first preference always! (Result 1)
- How to learn other preferences? *Induction!*

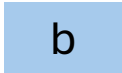
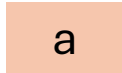
a: $1 \succ_c 2 \succ_a$ 

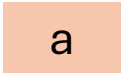
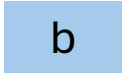
b: $2 \succ_b$  $\succ_b$ 

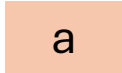
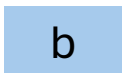
c: $1 \succ_c$  $\succ_c$ 

pairs:

(a,2),(b,1),(c,3)

1:  $\succ_1$  $\succ_1$ 

2:  $\succ_2$  $\succ_2$ 

3:  $\succ_3$  $\succ_3$ 

Result 3: Propose&Reject

- Using a P&R algorithm where **dishonest agents propose**, everything can be learned in n^2 rounds.

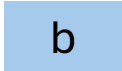
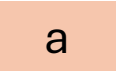
- We can learn the first preference always! (Result 1)
- How to learn other preferences? *Induction!*

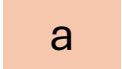
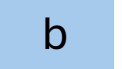
a: $1 \succ_c 2 \succ_a$ 

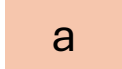
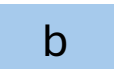
b: $2 \succ_b$  $\succ_b$ 

c: $1 \succ_c$  $\succ_c$ 

pairs:
(a,2),(b,1),(c,3)

1:  $\succ_1$  $\succ_1$ 

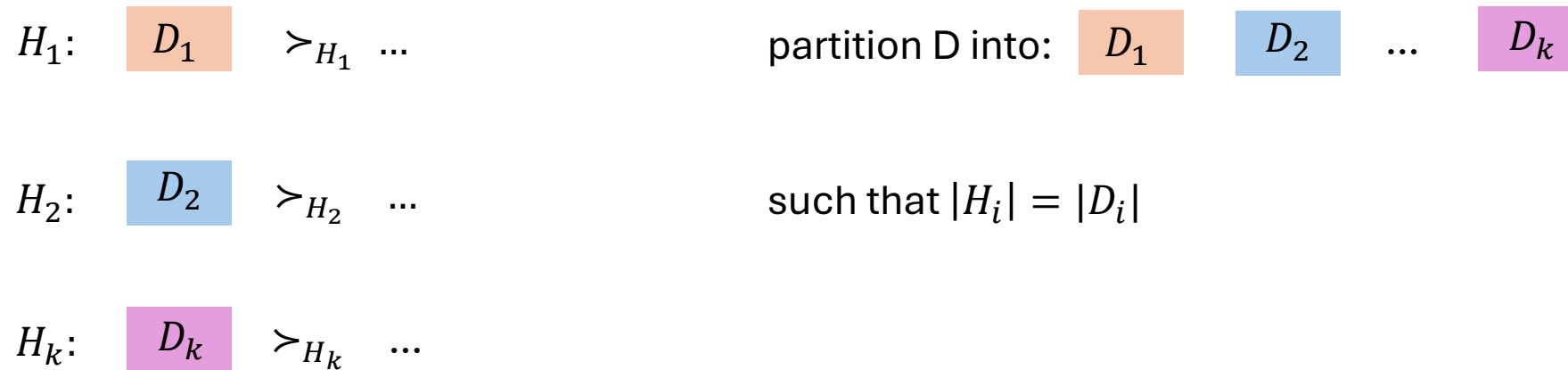
2:  $\succ_2$  $\succ_2$ 

3:  $\succ_3$  $\succ_3$ 

Targeted
Propose
Attack

Result 4: But also good news for P&R

- If honest side proposes, then P&R is privacy preserving.



Decentralized Algorithms

- P&R where **honest** side proposes can learn **all preferences** in one algorithm iteration with $2n$ communication rounds
- P&R where **dishonest** agents propose can learn **all preferences** in $n(n-1)$ algorithm iterations with at least **two** communication rounds per iteration

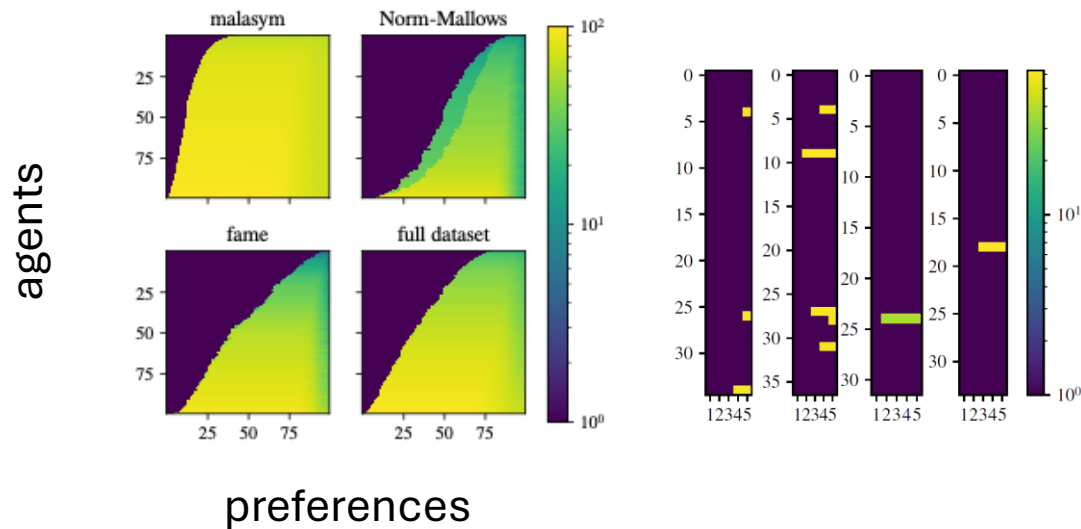
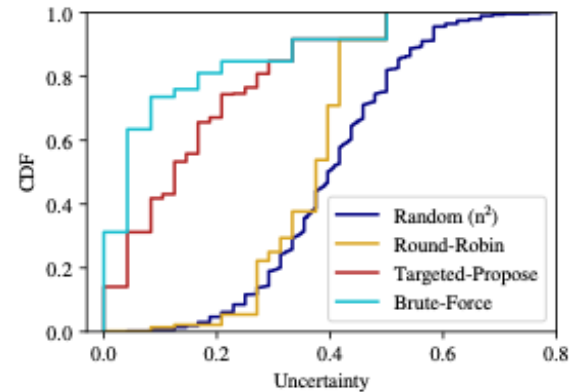
Simulations: Empirical Results

For different attack strategies:

- Random – try out random n^2 preference matrices
- Round Robin
- Brute Force
- Targeted Propose

Empirical Results

- Different strategies gain different information about preferences
- Uncertainty reduction = information gain



- Synthetic and real datasets (students)
- Distribution of how much can be learned about specific agents

Future Work

- Computational complexity
- Generalizations and case studies

Thank you!